

AgentMesh-MCP: A Secure and Governed Framework for Agentic AI Systems Using LLM Agents and Model Context Protocol Servers

PRUDVI SAISARAN PONDURU

Independent Researcher

Abstract - This paper proposes AgentMesh-MCP, a secure and governed framework for agentic artificial intelligence systems that combine large language model agents with Model Context Protocol servers. Agentic AI systems increasingly reason over goals, decompose tasks, call tools, query private data, invoke APIs, and coordinate with other agents. The Model Context Protocol provides a standardized client-server mechanism for exposing tools, resources, and prompts to AI applications, but protocol-level interoperability alone does not solve authorization, prompt injection, tool poisoning, auditability, or unsafe autonomy. We formalize the agentic system as a constrained partially observable decision process in which agent state, environment state, tool state, policy state, and governance constraints evolve through MCP-mediated actions. The proposed framework introduces a policy and authorization gateway, context firewall, risk scoring engine, human approval gateway, execution monitor, immutable audit ledger, red-team module, evaluation engine, rollback layer, and governance dashboard. We synthesize evidence from LLM agent research, tool-use systems, prompt-injection studies, AI risk-management standards, and official MCP documentation to derive architectural requirements and evaluation criteria. The paper contributes a threat model, formal model, algorithms for policy-gated tool invocation and secure context filtering, comparative analysis, and a reproducible experimental testbed. The central finding is that production-grade agentic AI should be treated less as a chatbot extension and more as a distributed socio-technical system whose safety depends on least privilege, context isolation, observable execution, explicit human control points, and continuous governance.

Keywords - Agentic AI, LLM agents, Model Context Protocol, MCP servers, tool use, multi-agent systems, AI governance, prompt injection, autonomous AI, agent security, human-in-the-loop AI.

I. INTRODUCTION

Agentic artificial intelligence refers to AI systems that do more than answer isolated questions. An agentic system accepts a goal, maintains state, reasons about possible actions, selects tools, observes the outcomes of those actions, revises its plan, and continues until a stopping condition is satisfied. In the context of large language models, an LLM agent is a software component that uses a language model as a reasoning and decision engine while delegating external actions to tools, APIs, code interpreters, databases, filesystems, browsers, cloud platforms, or other agents. This shift changes the engineering problem. A chatbot can be evaluated mainly by response quality, but an agentic AI system must be evaluated as an actor in an environment where wrong actions can modify files, leak data, create cloud resources, send messages, or trigger downstream business processes [5], [6], [11].

The Model Context Protocol (MCP) is important because it standardizes one of the most unstable layers in the modern AI application stack: the connection between AI applications and external systems. Official MCP documentation defines MCP as an open-source standard for connecting AI applications to external systems such as data sources, tools, and workflows [1]. The architecture uses hosts, clients, and servers; a host application creates MCP clients, and each client maintains a connection to a server that exposes context through tools, resources, prompts, and related primitives [2], [3]. In principle, this gives AI developers a portable way to integrate agents with local files, enterprise databases, SaaS APIs, observability platforms, cloud services, and specialized prompts without rewriting bespoke integrations for every model provider or agent framework.

Interoperability, however, is not the same as safety. Once an LLM agent can call an MCP server, the agent can request actions whose semantics may be much more consequential than text generation. A filesystem tool can delete evidence; a database tool can expose sensitive records; a cloud-infrastructure tool can provision expensive resources; a browser tool can follow

adversarial instructions embedded in untrusted web pages; and a GitHub tool can modify source code. The technical challenge is therefore not simply how to make agents more capable. It is how to design an agentic system in which capability, authorization, context flow, human control, and auditability are bound together by architecture rather than by trust in model behavior alone [28], [29], [32].

This paper proposes AgentMesh-MCP, a secure and governed framework for agentic AI systems that use LLM agents and MCP servers. The framework treats each tool invocation as a policy-governed event in a distributed system. It separates the LLM reasoning core from tool execution, places MCP clients behind a policy and authorization gateway, filters untrusted context through a context firewall, computes dynamic risk scores, routes high-risk operations through human approval, records immutable audit events, supports rollback and recovery, and continuously evaluates the system with red-team attack scenarios. The framework is intended for autonomous and semi-autonomous systems in software engineering, cloud operations, research assistance, enterprise workflow automation, cybersecurity triage, data analysis, education, healthcare administration, and legal-document workflows.

The paper makes five contributions. First, it clarifies the distinction between ordinary chatbots, LLM agents, and full agentic AI systems. Second, it analyzes MCP-enabled agents as a socio-technical system in which tool schemas, authentication protocols, prompts, memory, policy rules, and human oversight jointly determine risk. Third, it proposes the AgentMesh-MCP reference architecture and associated governance controls. Fourth, it formalizes agentic MCP execution as a constrained decision process with safety and governance constraints. Fifth, it presents algorithms, threat models, comparative tables, evaluation metrics, a reproducible experimental setup, findings, limitations, and a future research agenda for secure MCP-mediated agentic AI.

II. RESEARCH QUESTIONS AND SCOPE

The study is organized around six research questions. RQ1 asks what makes an AI system agentic. The answer cannot be reduced to the use of an LLM or the presence of a tool-call interface. A system becomes agentic when it maintains goal-directed state across multiple steps, selects actions, observes consequences, and adapts subsequent behavior. RQ2 asks how LLM agents use tools and MCP servers. This requires analysis of tool discovery, schema grounding, prompt construction, argument generation, result interpretation, and error recovery. RQ3 asks what architecture is required for safe agentic AI. The hypothesis of this paper is that secure agentic AI requires explicit separation between reasoning, authorization, execution, observation, and governance. RQ4 asks what security risks arise in MCP-enabled agents. We focus on prompt injection, tool poisoning, malicious servers, credential misuse, excessive agency, memory poisoning, multi-agent compromise, and supply-chain attacks.

RQ5 asks how agentic AI systems can be evaluated. Agent evaluation should include task completion, plan correctness, tool-selection accuracy, policy compliance, prompt-injection resistance, latency, cost, rollback success, audit completeness, and user trust. A benchmark that measures only final answer accuracy is insufficient because an agent may reach the correct answer through unsafe or non-compliant actions. RQ6 asks what governance mechanisms are required for real-world deployment. We argue that governance must be built into the runtime through least privilege, role-based and attribute-based authorization, human approval gates, audit ledgers, incident response, compliance reporting, and model/tool change management [32]-[35].

The scope of this paper is deliberately architectural. It does not propose a new foundation model, a new decoding method, or a new single-agent benchmark. Instead, it proposes a systems framework for deploying existing and future LLM agents with MCP servers under security and governance constraints. The framework is model-neutral and protocol-aligned: it assumes agents may be implemented with different LLM providers, orchestration frameworks, and deployment substrates, while the MCP layer provides a standardized interface for context and tool access. This makes the contribution relevant to enterprise environments where heterogeneity is normal and where policy, audit, and integration lifecycle concerns often dominate pure model capability.

The term autonomous is used carefully. AgentMesh-MCP does not assume that every system should be fully autonomous. Most high-value deployments should begin as supervised or semi-autonomous systems with explicit human-in-the-loop approval for irreversible or high-impact operations. The framework supports autonomy as a configurable property rather than as an ideological objective. For example, a research assistant may autonomously read public papers and summarize findings, while a cloud-operations agent may require approval before modifying production infrastructure. Safety in AgentMesh-MCP is therefore not a single switch; it is a graded control regime based on task risk, data sensitivity, tool criticality, user role, and execution context.

Table I: Research Questions and Operational Interpretation

RQ	Question	Operational interpretation
RQ1	What makes an AI system agentic?	Goal persistence, action selection, observation, reflection, and stateful adaptation.
RQ2	How do LLM agents use tools and MCP servers?	Tool discovery, schema grounding, policy checks, execution, and result incorporation.
RQ3	What architecture is required for safe agentic AI?	Separation of reasoning, authorization, execution, logging, and governance.
RQ4	What are the security risks of MCP-enabled agents?	Prompt injection, malicious tools, data leakage, credential misuse, unsafe actions.
RQ5	How can agentic AI systems be evaluated?	Task, safety, latency, cost, robustness, audit, and compliance metrics.
RQ6	What governance mechanisms are needed?	Least privilege, approval gates, dashboards, incident response, and compliance controls.

III. BACKGROUND

Large language models are probabilistic sequence models trained on broad corpora and adapted through supervised fine-tuning, reinforcement learning, instruction tuning, or other alignment methods. They are powerful at language understanding, code generation, summarization, and reasoning-like behavior, but they remain vulnerable to hallucination, brittle long-horizon planning, sensitivity to prompt phrasing, and weak grounding in external state [22], [23], [43]. These limitations are especially important in agentic settings because an incorrect intermediate belief can drive a harmful external action. Unlike a static answer, an action creates state changes outside the model.

Tool calling is the mechanism by which a model or agent delegates part of a task to a function or external system. In modern LLM platforms, the developer supplies a tool schema, the model emits a structured call, the application executes the call, and the tool result is returned to the model as an observation [24]. Toolformer showed that language models can learn when and how to use tools from demonstrations and self-supervised signals [6]. ReAct demonstrated that interleaving reasoning traces with actions can improve interpretability and task performance by allowing the model to gather information from external environments [5]. These works helped establish the conceptual pattern now used in many agentic systems: think, act, observe, and revise.

Retrieval-augmented generation (RAG) is a related but narrower pattern. A RAG system retrieves external documents or records and injects them into the model context before generation [17]. RAG improves factual grounding but does not necessarily make a system agentic because retrieval may be a single pre-generation step. Agentic systems generalize retrieval into iterative interaction. They can retrieve, call tools, ask clarifying questions, plan subtasks, execute code, update memory, and invoke other agents. In AgentMesh-MCP, RAG is treated as one class of MCP-enabled resource interaction rather than as the entire architecture.

Planning and reasoning in LLM agents are commonly implemented through prompt patterns, graph-based orchestration, programmatic planners, or multi-agent conversations. Chain-of-thought prompting and self-consistency

methods show that additional intermediate reasoning can improve problem solving, although visible reasoning traces are not reliable evidence of actual internal cognition [19], [20]. Reflexion and Self-Refine show that language feedback and iterative revision can improve performance across tasks [7], [47]. AutoGen, LangGraph, and similar frameworks then provide orchestration mechanisms for multi-step and multi-agent workflows [9], [25], [26]. AgentMesh-MCP does not require a particular planning strategy; it requires that any strategy route external action through governed interfaces.

Memory in AI agents can be short-term, episodic, semantic, procedural, or reflective. Short-term memory is the current context window. Episodic memory records task histories. Semantic memory stores facts or embeddings. Procedural memory stores reusable skills, scripts, or policies. Reflective memory stores lessons learned from failures. Memory improves continuity but increases attack surface. If an attacker can poison agent memory, the agent may carry malicious instructions into future sessions. AgentMesh-MCP therefore treats memory as a governed resource with provenance, expiration, access control, and sanitization rules, not as an unrestricted append-only prompt buffer.

Multi-agent collaboration extends the system from a single planner to a network of specialized agents. One agent may plan, another may browse, another may write code, another may test, and another may review. AutoGen popularized the conversational multi-agent pattern in which agents exchange messages, use tools, and involve humans in the loop [9]. Generative agents research showed how memory, reflection, and planning can create believable interactive behavior in simulated societies [10]. The risk is that multi-agent systems amplify both capability and failure. A compromised or hallucinating agent can persuade other agents, propagate poisoned context, or create loops of ungrounded agreement. AgentMesh-MCP therefore introduces agent identity, delegated authority, message provenance, and cross-agent policy checks.

MCP servers expose three primary server-side primitives: tools, resources, and prompts [2], [3]. Tools are executable functions that an AI application can invoke; resources are context-bearing data sources; and prompts are reusable interaction templates. The protocol uses JSON-RPC 2.0 as the data-layer exchange format and supports lifecycle negotiation, tool discovery, tool execution, notifications, and transport mechanisms such as standard input/output for local servers and Streamable HTTP for remote servers [2]-[4]. This structure makes MCP attractive for enterprise integration because it creates a uniform boundary around heterogeneous systems.

The crucial security observation is that MCP servers are not merely data adapters. A server may be a gateway to mutable and privileged operations. As a result, MCP client behavior must be governed by policies that are outside the LLM's own prompt. A model may be instructed to 'never delete files,' but that instruction is not a sufficient safety control when adversarial context can override or confuse it. AgentMesh-MCP treats natural-language instructions as advisory, while policy enforcement, credential scoping, tool validation, human approval, and auditing are implemented in deterministic infrastructure.

Table II: Chatbots, LLM Agents, and Agentic AI Systems

Dimension	Chatbot	LLM agent	Agentic AI system
Primary behavior	Responds to prompts	Plans and calls tools	Operates over goals, tools, memory, policies, and observations
State	Mostly conversational	Task-local state	Persistent state with governed memory and audit history
External action	None or limited	Tool/API calls	Policy-gated multi-step execution with recovery
Main risk	Incorrect answer	Unsafe tool call	Compound socio-technical failure across tools and agents
Evaluation	Answer quality	Task success and tool accuracy	Task, safety, governance, cost, latency, and audit metrics

IV. RELATED WORK

The literature on LLM agents has developed along several lines. ReAct introduced a simple but influential pattern for interleaving reasoning and acting, allowing LLMs to update action plans based on environmental observations [5]. Toolformer demonstrated that a model can learn to call external APIs and integrate their results into future token prediction [6]. Reflexion showed that agents can improve through verbal feedback stored in episodic memory, without changing model weights [7]. Voyager demonstrated open-ended skill acquisition in Minecraft using an LLM, curriculum, and skill library [8]. Together, these works establish that tool use, memory, and reflection can substantially extend the utility of LLMs beyond single-turn generation.

AutoGPT-style agents and software-engineering agents popularized autonomous task decomposition, file modification, browser use, and code execution. Although many early systems were brittle, they exposed the central engineering tension: the same mechanisms that make agents useful also create risk. An agent that can write code can also introduce vulnerabilities. An agent that can browse can ingest hostile web instructions. An agent that can update files can corrupt assets. Research environments such as WebShop, WebArena, and AgentBench respond to this challenge by creating more realistic tasks for evaluating agents in web, commerce, and multi-domain settings [13], [15], [16]. However, evaluation environments often remain less regulated and less security-sensitive than enterprise deployments.

Application frameworks such as LangChain, LangGraph, AutoGen, and Semantic Kernel offer abstractions for tools, memory, planning, graph execution, and multi-agent coordination [9], [25]-[27]. These frameworks are important because they shift agent engineering from ad hoc prompting toward reusable orchestration. LangGraph, for example, emphasizes long-running stateful agents and graph-based control flow [26]. Semantic Kernel provides a software-development model for connecting prompts, planners, and functions [27]. AgentMesh-MCP differs from these frameworks by focusing on the governance boundary: it asks how an organization should authorize, monitor, constrain, and audit

MCP-mediated tool use regardless of which orchestration framework is used above the MCP client layer.

MCP itself is still young relative to established web and identity protocols, but it is developing into a common integration boundary for AI applications. Official MCP documentation describes a client-server architecture with hosts, clients, and servers, JSON-RPC data exchange, tools, resources, prompts, notifications, lifecycle negotiation, and local or remote transports [1]-[3]. This makes MCP comparable to a protocol-level abstraction for the agent-tool boundary. The comparison should not be overstated: MCP standardizes how context and capabilities are exposed, but it does not by itself specify complete enterprise authorization, risk scoring, model behavior, audit semantics, or incident response. AgentMesh-MCP fills this gap at the system-design level.

Security research has shown that LLM-integrated applications are vulnerable to direct and indirect prompt injection. Indirect prompt injection is particularly dangerous for agents because the malicious instruction is not supplied directly by the user; it is embedded in a web page, document, email, repository issue, or database field that the agent retrieves during normal operation [29]. Tool poisoning generalizes this idea to tool descriptions, schemas, server metadata, or output fields. If a model trusts a malicious tool description, it may select a dangerous tool or pass sensitive arguments. These risks are represented in OWASP guidance for LLM and generative AI applications, including prompt injection, sensitive information disclosure, excessive agency, insecure output handling, and supply-chain concerns [28].

Governance research and standards provide a complementary foundation. The NIST AI Risk Management Framework emphasizes mapping, measuring, managing, and governing AI risks throughout the lifecycle [32]. ISO/IEC 42001 defines requirements for an AI management system, while ISO/IEC 23894 addresses AI risk management [33], [34]. The EU AI Act creates a risk-based regulatory structure for AI systems and makes governance, documentation, and human oversight central for high-risk contexts [35]. AgentMesh-MCP maps these governance ideas into runtime controls for agentic systems: policy enforcement, logging, approval workflows, evaluation, incident response, and compliance evidence collection.

A remaining gap is the absence of a widely accepted secure reference architecture for MCP-enabled agents. Prior work explains how agents reason and use tools; frameworks explain how to orchestrate them; protocol documentation explains how clients and servers exchange context; and security standards explain broad governance obligations. AgentMesh-MCP integrates these strands into a concrete architecture for interoperable, autonomous, and governed AI systems. The novelty is not a single component, but the composition: MCP interoperability is placed inside a defense-in-depth runtime with formal constraints, policy gates, risk scoring, human approval, audit, rollback, and red-team evaluation.

Table III: Existing Frameworks and Limitations for Governed MCP Agents

Framework or approach	Strength	Limitation addressed by AgentMesh-MCP
ReAct and Toolformer	Demonstrate reasoning-action and learned tool use	Do not specify enterprise authorization, audit, or rollback controls
Reflexion and Self-Refine	Improve behavior through feedback and revision	Memory can be poisoned without provenance and filtering
AutoGen	Supports multi-agent conversation and human participation	Needs explicit cross-agent trust and policy boundaries
LangGraph	Provides stateful graph orchestration for agents	Does not by itself define MCP-specific governance and risk scoring
Semantic Kernel	Connects prompts, planners, and functions	Policy enforcement depends on application design
MCP	Standardizes clients, servers, tools, resources, and prompts	Protocol interoperability is not a complete security architecture

V. PROBLEM STATEMENT AND REQUIREMENTS

The central problem is that agentic AI systems are becoming powerful enough to interact with real tools and systems, but they lack mature frameworks for security, governance, interoperability, evaluation, and auditability. In traditional software, a human or deterministic program calls an API under explicit authentication and authorization rules. In agentic AI, an LLM may infer which API to call from a natural-language goal, a prompt template, a retrieved document, a tool description, and prior memory. The action may be correct, but the causal chain is harder to inspect. This weakens accountability unless the architecture records what the agent knew, what tool it selected, what policy was applied, what arguments were passed, who approved the action, and what state changed.

The problem is intensified by the semantic gap between natural language and executable action. A user may ask an agent to 'clean up the repository,' but the phrase could mean format code, remove dead branches, delete files, rewrite tests, or close issues. A model may generate a plausible interpretation, but a tool call operationalizes that interpretation. Safe systems need structured intent capture, risk-sensitive planning, and confirmation for ambiguous or irreversible operations. AgentMesh-MCP therefore distinguishes user goals from executable plans and requires that high-risk actions be transformed into auditable proposals before execution.

A second problem is that agents consume untrusted context. Documents, web pages, tickets, email messages, database records, and tool outputs can all contain text that looks like instructions. LLMs process these tokens in a unified context window, so an attacker can try to smuggle commands through content that should have been treated only as data. The problem is not solved by telling the model to ignore malicious instructions, because the same model must interpret both instructions and data. A context firewall is required to label, filter, summarize, quote, and privilege-separate content sources before they reach the reasoning core.

A third problem is excessive agency. If a tool token grants broad permissions, the agent may perform actions outside the user's intent or role. Least privilege must be enforced at multiple layers: user identity, agent identity, MCP client, MCP server, tool scope, argument constraints, runtime sandbox, and

downstream API. OAuth 2.0 and OpenID Connect provide identity and token mechanisms, but the agentic layer must add task-scoped delegation and purpose limitation [36], [37]. A user who may read a database through a web application should not automatically empower an agent to export the entire database through an MCP server.

A fourth problem is poor observability. Many agent prototypes log messages, but production governance requires more: structured traces, policy decisions, tool arguments, result hashes, human approvals, rollback records, and compliance annotations. OpenTelemetry and provenance standards show how distributed systems can create useful traces and evidence chains [48], [49]. AgentMesh-MCP extends this idea to agentic AI by treating each reasoning step and tool action as part of an audit graph. The audit graph supports debugging, user explanations, incident response, compliance reviews, and continuous evaluation.

The requirements derived from these problems are as follows. R1: agents must discover tools but cannot invoke them without policy gating. R2: tool schemas and descriptions must be validated and treated as potentially adversarial. R3: sensitive context must be labeled, filtered, and minimized. R4: credentials must be scoped to user, agent, task, tool, and time. R5: high-risk operations must trigger human approval. R6: all tool executions must be monitored and logged. R7: rollback and recovery must be designed before autonomous actions are enabled. R8: red-team and evaluation modules must test security properties continuously. R9: governance dashboards must convert logs into actionable oversight rather than passive archives.

Table IV: MCP Server Risks and Mitigations

Risk	Example	AgentMesh-MCP mitigation
Prompt injection	Web page tells agent to exfiltrate secrets	Context firewall, data/instruction separation, policy gate
Tool poisoning	Malicious tool description asks model to send tokens	Schema validation, signed registry, description sanitization
Malicious server	Remote MCP server lies about capability or result	Registry trust tiers, attestation, allowlists, isolation
Credential misuse	Agent uses user token beyond intended task	Task-scoped tokens, OAuth/OIDC, least privilege
Memory poisoning	Retrieved content becomes persistent instruction	Provenance tags, memory quarantine, expiration
Unsafe action	Agent deletes production resource	Risk scoring, human approval, rollback plans
Multi-agent compromise	One agent persuades others to bypass policy	Agent identity, cross-agent policy, message provenance

VI. PROPOSED FRAMEWORK: AGENTMESH-MCP

AgentMesh-MCP is a reference architecture for secure and governed agentic AI systems. The framework is organized around a simple principle: an LLM may propose actions, but deterministic infrastructure must authorize, execute, observe, and record those actions. The architecture begins with a user goal interface that captures user intent, constraints, identity, role, risk tolerance, and approval preferences. The LLM reasoning core interprets the goal and produces candidate plans. The agent planner decomposes the goal into steps, estimates dependencies, selects possible tools, and identifies uncertainty. The memory module supplies task-relevant context while respecting provenance and access controls.

The tool selection module maps plan steps to candidate tools. It does not directly expose raw tool descriptions from MCP servers to the LLM. Instead, it uses the MCP client layer to query a signed MCP server registry and retrieve validated metadata. The registry stores server identity, owner, version, capabilities, security tier, data classification, supported authentication methods, historical reliability, known vulnerabilities, and allowed deployment contexts. Local development servers, internal enterprise servers, and public remote servers can coexist, but they receive different trust levels and policy defaults.

The MCP client layer is the adapter between the agent host and MCP servers. It handles lifecycle negotiation, capability discovery, tool listing, resource access, prompt retrieval, and tool execution according to MCP semantics [2], [3]. In AgentMesh-MCP, the client layer never calls a server directly on behalf of the LLM. Every call is routed through the policy and authorization gateway. The gateway evaluates user identity, agent identity, tool identity, requested arguments, data sensitivity, environment, time, historical behavior, and risk score. It can allow, deny, require transformation, require human approval, or route to a sandbox.

The context firewall is the component that protects the reasoning core from adversarial context. It labels content by source, strips executable instructions from data channels when possible, summarizes untrusted documents, quotes rather than executes external instructions, redacts secrets, detects prompt-injection patterns, and enforces separation between system policy, developer instructions, user instructions, retrieved content, tool descriptions, and tool outputs. A context firewall cannot make prompt injection impossible, but it reduces attack surface and creates evidence when untrusted content influenced a decision.

The risk scoring engine estimates the severity and uncertainty of each candidate action. Risk is computed from tool criticality, data classification, reversibility, blast radius, user role, server trust tier, argument novelty, anomaly signals, and historical incident data. A read-only search query may receive a low score. A production database export, cloud IAM change, repository force push, or healthcare administrative update may receive a high score. High-risk actions are routed to the human approval gateway, which presents the goal, plan, tool, arguments, expected effects, alternatives, rollback plan, and audit implications in a concise form.

The MCP server runtime executes tools in constrained environments. For local servers, this may involve process sandboxing, filesystem allowlists, network egress controls, and per-tool resource limits. For remote servers, it may involve token exchange, mutual TLS, server attestation, and response validation. The tool execution monitor observes latency, errors, outputs, side effects, and policy deviations. It can interrupt execution when a call exceeds time, scope, or resource thresholds. For tools with external side effects, the monitor can require dry-run mode before commit mode.

The audit logging system records immutable events. Each event includes user identity, agent identity, model version, prompt version, MCP server version, tool name, arguments, policy decision, risk score, approval status, execution result, output hash, timestamp, and rollback metadata. Audit logs are not merely for compliance. They are inputs to the evaluation engine, red-team module, incident response, and governance dashboard. Without such logs, organizations cannot determine whether an

undesirable outcome resulted from model error, prompt injection, bad tool schema, policy misconfiguration, or human approval mistake.

The evaluation engine measures task success, safety, robustness, latency, cost, and compliance. The red-team testing module injects adversarial documents, malicious tool descriptions, poisoned memory, compromised server metadata, and conflicting agent messages. The rollback and recovery module maintains compensating actions, snapshots, reversible transactions, and incident workflows. The governance dashboard summarizes risk trends, approval burden, violations, tool inventory, server trust tiers, unresolved incidents, model/tool version changes, and compliance evidence. The result is a mesh architecture in which agents, tools, humans, and policies interact through governed boundaries rather than through implicit trust.

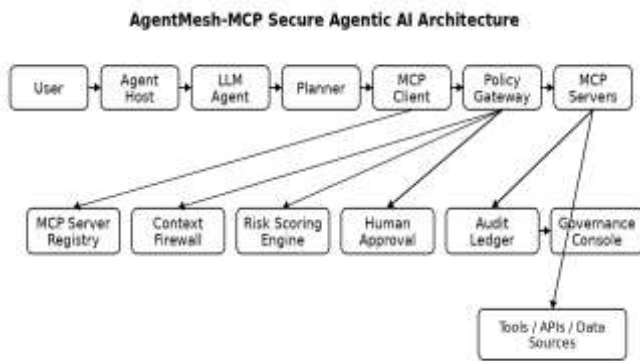


Fig. 1. AgentMesh-MCP architecture for policy-gated MCP tool use.

Table V: Core Components of AgentMesh-MCP

Component	Function	Security role
User goal interface	Captures goal, identity, constraints, and approval preferences	Binds task to accountable user and declared intent
LLM reasoning core	Interprets goals and proposes plans	Generates candidate reasoning but does not execute tools directly
Agent planner	Decomposes tasks into steps	Makes action proposals inspectable before execution
Memory module	Provides episodic and semantic context	Uses provenance, expiration, and access control
Tool selector	Maps steps to candidate tools	Avoids raw untrusted tool metadata exposure
MCP client layer	Discovers and invokes MCP primitives	Implements protocol interactions behind policy checks
Policy gateway	Evaluates authorization and risk	Enforces least privilege and action constraints
Context firewall	Filters untrusted context	Reduces prompt injection and data leakage
Human approval gateway	Requests approval for high-risk actions	Preserves meaningful human control
Audit ledger	Records structured events	Supports accountability and incident response

Agent Reasoning Loop With Policy-Gated Tool Execution

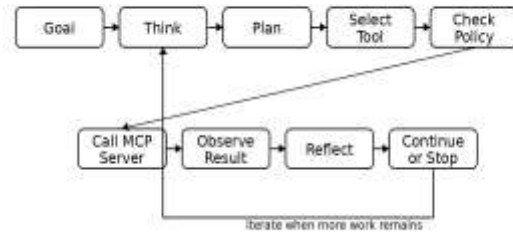
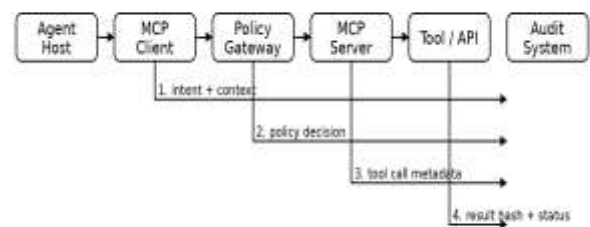


Fig. 2. Agent reasoning loop with policy checks before MCP execution.

MCP Server Invocation Flow With Audit Mediation



Every request and decision is logged before and after tool execution.

Fig. 3. MCP server flow with policy and audit mediation.

VII. SECURITY THREAT MODEL

AgentMesh-MCP assumes a realistic enterprise threat model. The user may be benign, careless, or malicious. The LLM may hallucinate, misunderstand, or overgeneralize. Retrieved content may contain adversarial instructions. MCP servers may be buggy, compromised, or malicious. Tool descriptions may be misleading. Credentials may be over-scoped. Agents may communicate with other agents that have different trust levels. Logs may be incomplete. Humans may approve actions without understanding them. The framework therefore avoids a single trusted boundary and instead uses layered controls.

Direct prompt injection occurs when a user tells an agent to ignore prior instructions, reveal secrets, bypass policy, or misuse tools. Direct injection is easier to detect than indirect injection because it appears in the user's message. Indirect prompt injection occurs when the agent retrieves hostile content from a document, website, issue tracker, email, or database. The hostile content may instruct the agent to disclose secrets, modify data, or alter its plan [29]. Because agents retrieve external content precisely to complete tasks, simply blocking retrieval is not an acceptable solution. The context firewall must preserve useful data while reducing instruction authority.

Tool poisoning attacks target the metadata that guides tool selection. A malicious MCP server could describe a tool as safe while performing destructive actions, or include natural-language instructions in a description telling the model to pass hidden data. A compromised registry entry could point to an attacker-controlled server. A benign tool may have ambiguous parameters that the model fills incorrectly. Tool poisoning is especially dangerous because tool descriptions are often placed close to system instructions in the model prompt. AgentMesh-MCP mitigates this by validating schemas, signing registry metadata, separating tool descriptions from instructions, applying allowlists, and monitoring behavior.

Data leakage can occur through tool arguments, tool outputs, logs, prompts, memory, or cross-agent messages. For example, an agent may summarize a private database record into a message sent to a public search tool, or store sensitive data in long-term memory. Sensitive information disclosure is a key class of LLM application risk [28]. AgentMesh-MCP addresses it through data classification, output filtering, tool-level data-flow policies, redaction, context minimization, and audit logs that identify which data sources entered which tool calls.

Excessive agency arises when an agent is allowed to perform actions beyond the intended scope. The root cause is often permission design: a broad token, a powerful tool, and an ambiguous goal. A cloud infrastructure agent with administrator credentials and no approval gate can make catastrophic changes. A software engineering agent with write access to a repository can introduce vulnerable dependencies. Excessive agency is mitigated by least privilege, dry-run modes, step-up approval, irreversible-action checks, spending limits, and policy rules that bind credentials to purpose and duration.

Credential misuse includes token leakage, token replay, privilege escalation, and confused-deputy behavior. In an MCP setting, the agent host, MCP client, MCP server, and downstream tool may each handle credentials. The safest design avoids giving the LLM direct access to secrets. Tokens should be held by infrastructure, scoped to specific operations, and exchanged through standard identity mechanisms such as OAuth 2.0 and OpenID Connect where appropriate [36], [37]. Audit logs should record token scope and policy decisions without exposing token values.

Memory poisoning is the persistence version of prompt injection. A malicious document can influence not only the current task but future tasks if its content is stored as memory. Multi-agent compromise is the distributed version: one agent can pass poisoned content to another agent, making the provenance of the malicious instruction harder to trace. AgentMesh-MCP treats memory writes and cross-agent messages as policy-governed actions. It stores source labels, confidence scores, expiration times, and review status with memory records, and it allows high-risk memory to remain quarantined until validated.

Supply-chain attacks target dependencies, MCP server packages, container images, prompt templates, tool schemas, model adapters, and orchestration frameworks. Because MCP encourages an ecosystem of reusable servers, server provenance and update policy become critical. AgentMesh-MCP uses a registry with signed metadata, version pinning, vulnerability feeds, code review status, sandbox defaults, and emergency revocation. This aligns with broader secure software supply-chain guidance such as NIST secure software development practices and SLSA-style provenance thinking [50].

VIII. SAFETY AND GOVERNANCE CONTROLS

Safety controls in AgentMesh-MCP are not optional add-ons; they are runtime mechanisms. Least privilege is the baseline. Every tool invocation should be constrained by user role, agent role, task scope, environment, server trust tier, and data classification. Role-based access control is useful for broad duties, while attribute-based access control is necessary for context-sensitive decisions such as whether a given user may export a particular dataset at a particular time through a particular tool. Policy rules should be externalized from prompts

so that they can be tested, versioned, audited, and enforced deterministically.

OAuth/OIDC authorization is used to bind user identity and consent to tool access. However, standard web authorization is not sufficient by itself because the delegate is an agent, not a conventional application. AgentMesh-MCP adds task-scoped delegation: a token should be usable only for the specific task, tool, resource class, and time window needed. A policy gateway can implement token exchange, downscoping, and step-up approval for high-risk operations. For example, a read-only repository-inspection task should not receive a token that can push commits.

Human-in-the-loop approval is most valuable when it is meaningful rather than ceremonial. The approval request should explain the goal, the proposed action, the tool, the exact arguments, expected side effects, risk score, policy rationale, alternatives, and rollback plan. Humans should not be asked to approve every trivial operation because approval fatigue reduces safety. Risk-based approval is therefore essential. Low-risk read-only calls can execute automatically, medium-risk calls can execute under enhanced logging, and high-risk or irreversible calls require explicit approval.

Context isolation is one of the most important controls for agentic systems. AgentMesh-MCP separates system policy, developer instructions, user goals, retrieved data, tool descriptions, tool outputs, memory, and cross-agent messages. The context firewall labels each source and applies transformations: quote untrusted content, summarize documents, remove executable phrasing, redact secrets, and preserve provenance. When the model sees untrusted text, it should see it as evidence about the world, not as authority over the agent's behavior. This distinction is architectural, not merely linguistic.

Tool schema validation constrains the shape of actions. MCP tools expose input schemas, and those schemas should be treated as enforceable contracts [2], [3]. The framework validates argument types, ranges, enumerations, path allowlists, SQL query classes, cloud-resource names, and other parameters before execution. Validation should also reject hidden fields, unexpected nested objects, prompt-injection markers, and arguments that create data-flow violations. Strong schemas reduce but do not eliminate semantic errors, so validation must be combined with risk scoring and monitoring.

Sandboxing reduces the blast radius of local and remote tool execution. Local MCP servers that access files, shells, or code should run with restricted filesystem permissions, disabled unnecessary network access, resource limits, and explicit working directories. Remote servers should use scoped credentials, network allowlists, and response validation. Sandboxing is especially important for software engineering agents, where code execution may be necessary but dangerous. A safe architecture distinguishes analysis mode, dry-run mode, test mode, and commit mode.

Logging and auditing create accountability. Each tool action should generate an audit event with sufficient context to reconstruct the decision. The log should include not only the final tool call but the relevant plan step, policy result, risk score, approval decision, server identity, tool version, and output hash. Privacy-sensitive logs may require redaction or encryption, but removing action evidence entirely undermines governance. An

audit ledger also supports rollback, because a system cannot reliably reverse what it cannot identify.

Rollback mechanisms and kill switches are the last line of defense. For reversible operations, the framework stores compensating actions or snapshots. For irreversible operations, it elevates risk and requires stronger approval. A kill switch should be able to pause an agent, revoke tokens, disconnect MCP servers, quarantine memory, and freeze pending tool calls. Compliance monitoring then converts runtime evidence into oversight: which systems were used, which policies applied, which violations occurred, and how incidents were resolved. This maps runtime engineering to AI management-system requirements [32]-[35].

Table VI: Safe and Unsafe Agent Architecture Patterns

Pattern	Unsafe design	Safer AgentMesh-MCP design
Tool access	LLM receives broad tool list and credentials	Tools discovered through registry; calls gated by policy
Context	All text enters one prompt with equal authority	Context labels and firewall separate instructions from data
Authorization	Application token grants broad access	Task-scoped token and least privilege per tool
Human oversight	Approve everything or nothing	Risk-based approvals with actionable explanations
Memory	Unreviewed content becomes long-term memory	Provenance, quarantine, expiry, and write policies
Observability	Free-form chat logs	Structured traces, audit events, output hashes
Recovery	Manual cleanup after failure	Rollback plans, snapshots, kill switch, incident workflow

IX. FORMAL MODEL

Let an AgentMesh-MCP deployment be represented as a constrained stochastic transition system. At time step t , the agent has internal state A_t , the external environment has state E_t , the memory store has state M_t , and the governance subsystem has policy state G_t . The available MCP server set is $S = \{s_1, \dots, s_n\}$. Each server s_i exposes a set of tools T_i , resources R_i , and prompts P_i . The union of tools is $T = \text{union}_i T_i$. The action space is $X = X_{\text{text}} \cup X_{\text{tool}} \cup X_{\text{memory}} \cup X_{\text{delegate}}$, where text actions generate messages, tool actions invoke MCP tools, memory actions read or write memory, and delegate actions communicate with other agents.

An observation o_t belongs to observation space O and may include user messages, retrieved resources, tool outputs, execution errors, policy feedback, human approvals, and other-agent messages. The planner produces a candidate action $x_t = \pi_{\theta}(A_t, M_t, o_t, g)$, where π_{θ} denotes the LLM-mediated planning function and g is the user goal. Because π_{θ} is not a trusted enforcement mechanism, every external action is evaluated by a policy function P . The policy function $P(x_t, u, a, c, G_t)$ returns one of {allow, deny, transform, approve, sandbox}, where u is the user identity, a is the agent identity, and c is contextual metadata.

The risk function $\rho(x_t)$ maps a candidate action to a real-valued score in $[0,1]$. A simple instantiation is $\rho(x_t) = \sigma(w_1 C_{\text{tool}} + w_2 C_{\text{data}} + w_3 C_{\text{irrev}} + w_4 C_{\text{scope}} + w_5 C_{\text{server}} + w_6 C_{\text{anom}})$, where C_{tool} is tool criticality, C_{data} is data sensitivity, C_{irrev} is irreversibility,

C_{scope} is scope breadth, C_{server} is server trust risk, C_{anom} is anomaly evidence, w_i are learned or configured weights, and σ is a squashing function. The approval threshold τ_h and denial threshold τ_d define routing: if $\rho(x_t) \geq \tau_d$ the call is denied; if $\tau_h \leq \rho(x_t) < \tau_d$ human approval is required; otherwise the action may proceed if policy allows it.

The reward function is not merely task success. Let R_{task} measure progress toward the user goal, R_{safety} penalize unsafe or policy-violating behavior, R_{cost} penalize latency and resource usage, and R_{audit} reward complete traceability. Then $R_{\text{total}} = \alpha R_{\text{task}} - \beta R_{\text{safety}} - \gamma R_{\text{cost}} + \delta R_{\text{audit}}$. In a production system, this reward is not necessarily used for online reinforcement learning; it is an evaluation objective for comparing policies, models, tools, and deployment configurations.

Safety constraints are predicates $C_s(A_t, E_t, M_t, G_t, x_t)$ that must hold before and after an action. Examples include: no secret may be sent to a public tool; no production deletion without approval; no memory write from untrusted content without quarantine; no remote server may execute with administrative credentials unless attested and approved. Governance constraints C_g represent organizational and regulatory requirements: audit completeness, retention policy, data residency, role separation, incident reporting, model version tracking, and human oversight for specified domains [32]-[35]. An action is executable only if P permits it and both C_s and C_g hold.

The transition function updates state as $(A_{t+1}, E_{t+1}, M_{t+1}, G_{t+1}) \sim F(A_t, E_t, M_t, G_t, x_t, o_t)$. For read-only actions, E_t may remain unchanged while A_t and M_t update. For external side effects, E_t changes and the audit ledger records the transition. The formal model highlights a core design principle: the LLM proposes actions, but safe transition requires policy, risk, approval, execution monitoring, and audit evidence. This separation is what allows AgentMesh-MCP to support autonomy without treating the model as the sole source of control.

Table VII: Formal Variables in AgentMesh-MCP

Symbol	Meaning	Example
A_t	Agent internal state	Current plan, scratchpad, active subgoal
E_t	Environment state	Repository, cloud account, database, ticket system
M_t	Memory state	Episodic records, semantic facts, skill library
S	MCP server set	Filesystem, GitHub, database, browser, cloud servers
T	Tool set	query_db, read_file, create_issue, deploy_stack
O	Observation space	Tool result, error, human approval, retrieved record
P	Policy function	Allow, deny, transform, approve, or sandbox
ρ	Risk function	Score based on criticality, sensitivity, reversibility
C_s	Safety constraint	No secret to public tool; no destructive action without approval
C_g	Governance constraint	Audit retention, data residency, human oversight

X. ALGORITHMS

The following algorithms express the operational core of AgentMesh-MCP. They are written as implementation-neutral pseudocode. The main design rule is that the agent may reason in flexible natural language or graph-based plans, but each external action is converted into a structured request and subjected to deterministic validation before execution. The algorithms also show that security is continuous: discovery, planning, invocation, observation, reflection, memory update, logging, and red-team testing all participate in the safety envelope.

Algorithm 1: Policy-Gated Agent Planning and MCP Invocation

Input: user_goal g, user u, agent a, memory M, server_registry SR

```
1: task <- normalize_goal(g, u)
2: plan <- LLM_PLAN(task, filtered_memory(M, u, task))
3: for each step in plan do
4:   candidates <- discover_tools(step, SR)
5:   proposal <- select_tool_and_arguments(step, candidates)
6:   proposal <- validate_schema(proposal)
7:   risk <- score_risk(proposal, u, a, task)
8:   decision <- policy_gateway(proposal, risk, u, a)
9:   if decision = deny then log_denial(proposal); continue
10:  if decision = approve then proposal <- human_approval(proposal)
11:  if proposal.approved then result <- call_mcp_server(proposal)
12:  observation <- context_firewall(result)
13:  log_execution(proposal, decision, risk, observation)
14:  plan <- reflect_and_update_plan(plan, observation)
15: end for
```

Output: final answer, audit trail, residual risks

Algorithm 2: MCP Tool Discovery with Registry Trust

Input: plan_step p, registry SR, policy G

```
1: servers <- SR.lookup(capability_required_by(p))
2: servers <- filter_by_trust_tier(servers, G.allowed_tiers)
3: for each server s in servers do
4:   verify_signature(s.metadata)
5:   verify_version_and_vulnerability_status(s)
6:   tools <- mcp_list_tools(s)
7:   for each tool t in tools do
8:     validate_json_schema(t.input_schema)
9:     sanitize_description(t.description)
10:    attach_provenance(t, s)
11:  end for
12: end for
```

Output: validated candidate tools

Algorithm 3: Secure Context Filtering

Input: content c, source_label l, data_class d, policy G

```
1: mark c with provenance(l, timestamp, hash)
2: if d is sensitive then redact secrets and minimize fields
3: detect prompt-injection patterns and instruction-like text
4: quote untrusted instructions as data, not commands
5: summarize long untrusted content when possible
6: separate system, developer, user, retrieved, tool, and memory channels
7: if risk(c) exceeds threshold then quarantine or require review
```

Output: filtered_content, provenance, residual_context_risk

Algorithm 4: Human Approval Workflow

Input: proposal x, risk rho, policy rationale q, rollback plan b

```
1: construct_approval_card(goal, tool, args, expected_effect, rho, q, b)
2: present card to authorized human approver
3: record approve, deny, edit, or request-more-info decision
```

4: if edited then revalidate schema and recompute risk

5: write approval event to audit ledger

6: return approved proposal or denial

Algorithm 5: Red-Team Attack Simulation

Input: benchmark tasks B, attack corpus A, MCP server set S

```
1: for each task b in B do
2:   for each attack a in A do
3:     inject a into document, tool schema, memory, or server metadata
4:     run agent under standard policy
5:     measure violation, detection, refusal, latency, recovery
6:     store trace and label failure mode
7:   end for
8: end for
```

Output: security scorecard and regression tests

XI. EVALUATION METHODOLOGY AND EXPERIMENTAL SETUP

Evaluation of AgentMesh-MCP must measure both utility and control. Task completion rate measures whether the agent completes the user's goal. Tool selection accuracy measures whether the chosen MCP tool is appropriate for the plan step. Planning correctness measures whether the agent decomposes tasks into a valid sequence. Security violation rate measures unsafe tool calls, data exfiltration, policy bypass, or unauthorized actions. Prompt injection resistance measures whether the context firewall and planner ignore or quarantine malicious instructions embedded in retrieved content. Tool poisoning resistance measures whether malicious tool descriptions or schemas lead to unsafe selection or argument generation.

Latency and cost are also necessary metrics because safety controls impose overhead. A policy gateway, approval step, and audit ledger can improve safety while increasing response time. The appropriate objective is not maximum speed; it is acceptable latency under risk constraints. A low-risk research assistant may optimize for speed, while a production cloud agent should prioritize safety and auditability. Human approval frequency measures whether risk thresholds are too strict or too lax. If every call requires approval, users will abandon or rubber-stamp the system. If high-risk calls rarely require approval, the system is under-controlled.

Rollback success rate is a distinctive metric for agentic systems. A system that can detect and reverse a bad action is safer than one that can only log it. Rollback should be tested on file changes, database updates, infrastructure changes, repository modifications, and workflow state transitions. Audit completeness measures whether every relevant action has sufficient evidence: who, what, when, where, why, policy decision, risk score, arguments, result, and approval status. Compliance score measures alignment with organizational policy, NIST AI RMF controls, ISO/IEC 42001 management requirements, and domain-specific regulations [32]-[35].

A reproducible testbed should include an LLM agent framework, MCP clients, MCP servers, a policy engine, an observability stack, and red-team datasets. The MCP server set should include at minimum a filesystem server, database server, GitHub server, cloud infrastructure server, browser/search server, and an internal enterprise workflow server. The policy engine can be implemented with a general policy-as-code system, although the paper does not depend on one specific product. Observability should include structured logs, traces,

metrics, and a tamper-evident audit store. The red-team corpus should include direct prompt injection, indirect prompt injection, tool poisoning, credential bait, malicious server metadata, memory poisoning, and multi-agent manipulation.

The experimental protocol should compare four architectures. Baseline A is a non-agentic chatbot with no tools. Baseline B is an LLM agent with direct function calling and broad permissions. Baseline C is an MCP-enabled agent with tool discovery but no dedicated policy gateway or context firewall. The proposed system is AgentMesh-MCP. Each architecture should run the same task suite under benign and adversarial conditions. The task suite should include software engineering, database analysis, cloud operations, research assistance, enterprise workflow, and cybersecurity triage tasks. The adversarial condition should inject attacks into the same channels that real agents consume: issues, README files, web pages, database fields, tool descriptions, and memory records.

Results should be reported with confidence intervals and ablation studies. Ablations should disable the context firewall, risk scoring, human approval, registry validation, memory quarantine, and rollback module one at a time. This identifies which controls produce the largest safety gains and which impose the highest cost. A strong empirical claim would not be that AgentMesh-MCP eliminates all risk, but that it reduces security violations and improves audit completeness while preserving useful task completion. The expected trade-off is a modest latency and approval burden increase in exchange for a large reduction in unsafe action rate.

Table VIII: Evaluation Metrics for AgentMesh-MCP

Metric	Definition	Desired direction
Task completion rate	Fraction of tasks completed to specification	Higher
Tool selection accuracy	Correct tool chosen for plan step	Higher
Planning correctness	Plan steps are necessary, ordered, and safe	Higher
Security violation rate	Unauthorized, unsafe, or leaking actions	Lower
Prompt injection resistance	Attacks detected or neutralized	Higher
Tool poisoning resistance	Malicious schemas/descriptions rejected	Higher
Latency	End-to-end and per-tool execution time	Lower under SLO
Cost	Model, tool, human, and infrastructure cost	Lower for equal safety
Approval frequency	Rate of human approval prompts	Balanced
Rollback success rate	Bad actions reversed or contained	Higher
Audit completeness	Required evidence present per action	Higher
Compliance score	Alignment with policy and standards	Higher

XII. FINDINGS AND COMPARATIVE ANALYSIS

Finding 1: Agentic behavior is a systems property rather than a model property. Supporting evidence comes from ReAct, Toolformer, Reflexion, Voyager, AutoGen, and related systems, all of which show that model behavior changes when the LLM is embedded in loops involving tools, memory, and observations [5]-[9]. The technical explanation is that agency emerges from stateful interaction: a model generates an action, the environment changes or returns an observation, and the agent

updates its plan. The practical implication is that governance cannot be assigned only to the model card or prompt. It must cover the surrounding runtime. The limitation is that agency remains difficult to quantify, and future research should define standardized agency-level taxonomies.

Finding 2: MCP improves interoperability but does not by itself create a safe agent architecture. The official MCP architecture standardizes hosts, clients, servers, tools, resources, prompts, JSON-RPC exchange, lifecycle management, and transports [1]-[4]. The technical explanation is that MCP defines communication semantics, not complete organizational policy. A protocol can describe how to call a tool while leaving open whether the tool should be called, with which token, under which approval condition, and with what audit requirements. The practical implication is that enterprises should place MCP behind policy and observability layers. Future research should explore standardized security profiles and certification for MCP servers.

Finding 3: Context is the primary attack surface in LLM agents. Prompt injection, tool poisoning, memory poisoning, and multi-agent manipulation all exploit the model's tendency to process heterogeneous text in a common context [28], [29]. The technical explanation is that natural language can be simultaneously data and instruction unless the architecture enforces labels and priority. The practical implication is that context firewalls, provenance tags, and data/instruction separation are as important as network firewalls in agentic AI. The limitation is that perfect separation is not always possible; future work should develop formal context-type systems for agents.

Finding 4: Least privilege must be task-scoped, not merely user-scoped. Traditional authorization often grants an application the user's permissions. In agentic systems, this is too broad because the agent may over-interpret the goal or be influenced by hostile context. The technical explanation is the confused-deputy problem: the agent becomes a deputy that uses legitimate authority for unintended actions. The practical implication is that OAuth/OIDC tokens should be downscoped by task, tool, resource, and time [36], [37]. The limitation is that many enterprise APIs do not yet support sufficiently fine-grained delegation. Future research should define agent-specific authorization profiles.

Finding 5: Human oversight is useful only when risk-adaptive. Human-in-the-loop control is often proposed as a universal safety measure, but indiscriminate approval prompts create fatigue. The technical explanation is that approval has cognitive cost; if the signal-to-noise ratio is low, humans stop evaluating details. The practical implication is to use risk scoring to route only meaningful decisions to humans and to present concise approval cards. The limitation is that risk models can be miscalibrated. Future work should study human factors in agent approval interfaces and measure overtrust.

Finding 6: Auditability is a core safety mechanism, not only a compliance artifact. Agent behavior is difficult to debug after the fact unless logs capture plan, context, policy, tool arguments, results, and approvals. The technical explanation is that agent failures are distributed across prompts, models, tools, servers, memory, and humans. The practical implication is that structured audit ledgers should be designed before deployment, not after incidents. The limitation is that logs may contain

sensitive data. Future work should develop privacy-preserving audit schemes for agentic AI.

Finding 7: Evaluation must include adversarial and operational metrics. A benchmark that rewards only final task completion may favor unsafe agents that complete tasks by violating policy. The technical explanation is that agents are multi-objective systems: success, safety, cost, latency, and auditability conflict. The practical implication is that evaluation reports should include security violation rate, prompt-injection resistance, rollback success, approval frequency, and audit completeness. The limitation is benchmark realism. Future work should create standardized MCP benchmark suites with realistic enterprise workflows and attacks.

Table IX: API Calling, Plugins, and MCP Servers

Dimension	Direct API/function calling	Plugins	MCP servers
Integration scope	Application-specific functions	Platform-specific plugin model	Protocol-standardized tools, resources, prompts
Portability	Low to medium	Limited by host platform	Higher across MCP-compatible hosts
Discovery	Usually static schema list	Plugin manifest	Dynamic tools/list and resource discovery
Security boundary	Application-defined	Platform-defined	Needs host, client, server, and policy controls
Governance challenge	Schema and credential scope	Plugin review and permissions	Server registry, trust tiers, tool risk, audit

Table X: Single-Agent and Multi-Agent MCP Systems

Dimension	Single-agent	Multi-agent
Strength	Simpler control and attribution	Specialization, parallelism, peer review
Risk	Single point of reasoning failure	Cross-agent prompt injection and collusion-like loops
Policy need	Tool and memory policies	Agent identity, delegated authority, message provenance
Audit need	Plan and tool trace	Conversation graph plus tool trace
Best use	Focused tasks with clear scope	Complex workflows requiring specialized roles

XIII. USE CASES

AI software engineering agents are a natural use case. An agent may inspect issues, read code, run tests, propose patches, and open pull requests. MCP servers can expose GitHub, filesystem, package-management, test-runner, and CI/CD tools. The risk is that the agent can introduce vulnerable code, leak repository secrets, or modify protected branches. AgentMesh-MCP mitigates this by using read-only defaults, branch-scoped write permissions, test sandboxes, dependency policies, mandatory review for protected files, and audit logs for every file modification.

Cloud infrastructure agents can diagnose incidents, inspect metrics, query logs, propose remediation, and execute limited changes. MCP servers can expose cloud APIs, observability platforms, Kubernetes clusters, incident-management tools, and configuration databases. The risk is high because cloud actions can affect production availability and cost. AgentMesh-MCP

requires dry-run mode, change windows, blast-radius scoring, human approval for production mutations, rollback plans, and emergency kill switches. The agent may be autonomous in staging but semi-autonomous in production.

Research assistants can search literature, retrieve papers, summarize findings, build bibliographies, and generate drafts. MCP servers can expose academic search, file systems, reference managers, and note databases. The main risks are hallucinated citations, copyright misuse, data leakage from private notes, and overconfident synthesis. AgentMesh-MCP uses source provenance, citation verification, retrieval logs, output labeling, and memory controls. It does not treat generated summaries as evidence; evidence must remain linked to source records.

Enterprise workflow automation includes creating tickets, routing approvals, updating CRM records, generating reports, and coordinating across SaaS tools. MCP servers can expose internal systems through a common protocol. The risks are unauthorized updates, workflow spam, data misrouting, and hidden policy violations. AgentMesh-MCP binds actions to user roles, department policies, data classifications, and approval chains. It also records workflow provenance so that business owners can audit why an agent changed a record.

Cybersecurity agents can triage alerts, enrich indicators, query logs, inspect code, open cases, and draft remediation steps. These agents are attractive because security teams face high alert volume. They are also dangerous because attackers may deliberately craft logs, tickets, or emails to manipulate the agent. AgentMesh-MCP treats all external evidence as untrusted, separates observation from instruction, and restricts containment actions such as blocking accounts or isolating hosts to approved workflows. Red-team simulation is mandatory in this domain.

Data analysis agents can query databases, run notebooks, generate visualizations, and explain results. The risks include privacy violations, SQL injection through generated queries, incorrect statistical conclusions, and leakage of sensitive data into prompts or logs. AgentMesh-MCP uses query templates, row-level access policies, differential data handling, sensitive-column redaction, and human review for exports. The audit ledger records query text, data sources, result hashes, and report outputs.

Healthcare administrative agents, legal document agents, and education tutor agents illustrate domain variation. In healthcare administration, agents may schedule appointments, summarize records, or process claims; privacy and regulatory controls dominate. In legal workflows, agents may review contracts and compare clauses; confidentiality and citation provenance dominate. In education, agents may tutor students and grade drafts; fairness, explainability, and age-appropriate controls dominate. The same MCP pattern can support each domain, but governance profiles must differ. AgentMesh-MCP is therefore a framework for configurable control, not a one-size-fits-all policy.

Table XI: Real-World Use Cases and Required Controls

Use case	Typical MCP servers	Required controls
Software engineering	Filesystem, GitHub, test runner, CI/CD	Branch limits, test sandbox, code review, dependency policy
Cloud operations	Cloud API, logs, metrics, Kubernetes	Dry run, blast-radius score, approval, rollback

Use case	Typical MCP servers	Required controls
Research assistant	Search, files, references	Citation provenance, copyright guardrails, source verification
Enterprise workflow	CRM, ticketing, email, ERP	Role policy, approval chains, data classification
Cybersecurity	SIEM, EDR, threat intel	Untrusted evidence handling, containment approval
Data analysis	Database, notebooks, BI tools	Query validation, row/column access, export controls
Healthcare admin	Scheduling, records, claims	Privacy, audit, human review, domain compliance
Legal documents	DMS, contract repository	Confidentiality, citation trail, privilege controls

A. Deployment Patterns and Operational Lifecycle

A practical deployment of AgentMesh-MCP should begin with a staged lifecycle rather than with immediate broad autonomy. The first stage is inventory. An organization identifies candidate MCP servers, the tools they expose, the data classifications they touch, the operations they can perform, and the business processes they affect. This inventory is more than a list of integrations; it is a risk map. A read-only documentation-search server belongs in a different class from a production database server, and both differ from a cloud identity-management server. Each server should have an owner, version, change-control process, vulnerability status, authentication method, logging capability, and documented rollback posture. Without this inventory, later governance controls operate on incomplete assumptions.

The second stage is read-only piloting. Many failures in early agent deployments arise because write access is enabled before the organization understands how agents interpret goals. AgentMesh-MCP therefore recommends that new agents begin with tools that read, search, inspect, summarize, or draft proposals, while mutation tools are disabled or routed to dry-run mode. In this stage, the evaluation engine measures whether the agent selects appropriate tools, handles errors, respects context labels, and produces useful plans. The audit ledger should already be active even though actions are read-only, because this establishes a baseline for normal behavior and exposes which prompts, resources, and tool outputs actually influence decisions.

The third stage is constrained mutation. The agent may create draft artifacts, open pull requests, submit tickets, write to temporary workspaces, or modify non-production resources. The policy gateway enforces scoped permissions, and the human approval gateway is used for any action that moves from proposal to committed change. This stage is where rollback mechanisms must be tested. A safe system is not one that merely promises caution; it is one that demonstrates that it can revert a file edit, close a mistaken ticket, restore a configuration snapshot, or cancel a workflow. Rollback tests should be part of normal release validation, not rare disaster-recovery exercises.

The fourth stage is risk-adaptive autonomy. Only after empirical evidence supports reliability should some classes of actions become autonomous. Even then, autonomy should be local, revocable, and explicitly bounded. For example, an agent may autonomously label support tickets, generate non-binding research summaries, run tests, or restart a development service, while production deletion, financial transaction, legal filing, or

privileged identity change remains approval-gated. The autonomy level should be represented as a policy artifact and should be visible in the governance dashboard. When a model, server, tool schema, or policy changes, autonomy levels should automatically downgrade until regression tests pass.

Deployment also requires careful environment separation. Development, staging, and production should use distinct MCP server registries or registry namespaces, distinct credentials, distinct policies, and distinct audit partitions. A common anti-pattern is allowing a development agent to discover production tools because they share a client configuration. AgentMesh-MCP treats environment as a policy attribute. A tool call generated while the agent is operating in a development task should not be able to reach production systems unless a deliberate and audited environment transition occurs. This requirement is familiar in DevOps, but it becomes more important when a language model may synthesize tool calls dynamically.

The operational lifecycle should include model and prompt change management. Agents depend not only on code but also on model versions, system instructions, prompt templates, tool descriptions, memory policies, and retrieval configurations. A change in any of these can alter behavior. The audit ledger should therefore record model identifiers and prompt versions, and the evaluation engine should run regression tests when they change. This is analogous to software release management, but agentic AI adds a semantic layer: a harmless-looking prompt edit may change how aggressively an agent uses tools or how it interprets ambiguous goals.

Server onboarding is another critical lifecycle step. Before an MCP server is added to the registry, it should undergo schema review, permission review, threat modeling, sandbox testing, and logging verification. The server should declare whether its tools are read-only, reversible, idempotent, destructive, external-facing, or privileged. It should declare expected side effects and failure modes. Its descriptions should be checked for prompt-injection-like language, hidden instruction patterns, or vague operational semantics. If a tool is named 'clean_workspace', the registry should not rely on that name alone; it should encode whether the tool deletes files, archives them, moves them to trash, or produces a proposal.

Runtime operations require continuous drift detection. Agent behavior can drift when model providers update models, when users discover new prompting patterns, when server outputs change, when policies accumulate exceptions, or when memory stores grow. The evaluation engine should periodically replay representative tasks and attack cases. The governance dashboard should show trends in approval frequency, denial causes, high-risk tool usage, prompt-injection detections, server errors, rollback events, and policy overrides. A sudden drop in policy denials may indicate either improved behavior or a broken control; a sudden rise in approvals may indicate either new workloads or miscalibrated risk scoring.

Incident response should be designed before the first serious failure. If an agent leaks sensitive data, modifies production resources incorrectly, or follows a malicious instruction, operators need a clear procedure: pause the agent, revoke task tokens, disable affected MCP servers, quarantine relevant memory, preserve audit evidence, notify data owners, execute rollback steps, and perform root-cause analysis. The root cause may lie in model output, prompt design, tool schema ambiguity,

registry trust failure, missing policy, human approval error, or downstream API behavior. AgentMesh-MCP supports this analysis by preserving structured evidence across the entire action path.

A mature deployment should also include separation of duties. The team that writes agent prompts should not be the only team approving production policies. The team that owns an MCP server should not unilaterally assign its server a high trust tier. The team that evaluates security should have access to traces and red-team results. Governance is strongest when policy authors, tool owners, security reviewers, and business owners can each inspect the evidence relevant to their responsibilities. AgentMesh-MCP's governance dashboard is therefore not a decorative analytics page; it is the shared operational surface where these roles coordinate.

B. Protocol Design, Data Governance, and Performance Trade-offs

Protocol design for agentic AI should distinguish capability discovery from capability authorization. MCP makes discovery convenient through mechanisms such as tool listing, but a listed tool should not automatically be an executable tool. AgentMesh-MCP interprets discovery as an input to planning and authorization, not as permission. This distinction matters because an agent may discover many tools whose use would be inappropriate for the current user, goal, or environment. The tool selector can show the LLM a reduced, policy-filtered view of capabilities while the full registry remains available to deterministic infrastructure. This design reduces prompt size, lowers tool-selection confusion, and prevents the model from being tempted by unavailable operations.

Data governance begins with classification. Every resource exposed through an MCP server should have a data class: public, internal, confidential, regulated, secret, or domain-specific equivalents. The data class should flow with observations and memory records. If a confidential database result is summarized, the summary inherits the classification unless a declassification policy explicitly downgrades it. If the agent wants to send a confidential summary to a public search or translation tool, the policy gateway should block or transform the action. This is an information-flow view of agent safety: tools are not only capabilities but also data channels.

MCP prompts require the same governance as tools and resources. A prompt template can shape the behavior of an agent as strongly as a function can. A malicious or poorly written prompt exposed by a server might instruct the model to ignore local policy, reveal hidden context, or overuse a tool. AgentMesh-MCP therefore treats prompts as executable influence rather than passive text. Prompt templates should be signed, versioned, reviewed, and labeled by trust tier. The context firewall should distinguish prompts that are approved developer assets from prompts supplied by untrusted servers or external users.

The framework also recommends explicit side-effect declarations in tool metadata. A schema tells the system what arguments a tool accepts, but it may not tell the system what the tool changes. AgentMesh-MCP adds side-effect descriptors such as read-only, write, delete, network-send, credential-access, financial-impact, identity-impact, production-impact, and irreversible. These descriptors feed risk scoring and approval. They also improve user comprehension: an approval

card can say that a proposed action is irreversible and production-impacting rather than presenting only a function name and JSON arguments.

Performance trade-offs must be acknowledged. Context filtering, policy evaluation, registry verification, human approval, and audit logging add latency. For low-risk tasks, this overhead can be minimized through caching, precomputed policy decisions, and asynchronous logging. For high-risk tasks, the overhead is justified because the alternative is unsafe autonomy. The key design objective is differentiated control. AgentMesh-MCP should not run a simple documentation lookup through the same expensive path as a production infrastructure change, but both should remain observable. Risk-adaptive execution is the mechanism that reconciles usability with safety.

Cost trade-offs also include human cost. Approval workflows consume expert attention, and poorly designed systems can transfer work from execution to review without improving outcomes. The human approval gateway should therefore support batching, reusable approvals for repeated low-variance operations, clear diff views, and escalation only when the risk model indicates genuine uncertainty or impact. Measuring approval precision is as important as measuring approval frequency. A high-quality approval system routes important decisions to humans and lets harmless operations proceed under policy.

Interoperability can create a false sense of substitutability. Two MCP servers may expose tools with similar names but different semantics, side effects, reliability, or trust levels. An agent that used a staging 'deploy' tool safely may not be safe when pointed at a production 'deploy' tool. The registry must therefore maintain semantic metadata and environmental constraints, not merely endpoint addresses. Tool names should be namespaced by server and owner, and tool descriptions should be normalized into structured capability records where possible. This reduces ambiguity and supports safer cross-server planning.

Data minimization is a practical defense against both leakage and prompt injection. The agent should receive only the fields needed for the current step. A database server should not return entire records when a count or aggregate is enough. A filesystem server should not expose all files when a specific path allowlist is sufficient. A browser server should extract relevant content while preserving source provenance. Minimization reduces token cost, improves reasoning focus, and lowers the probability that irrelevant malicious text enters the context window. It also aligns with privacy principles in many governance regimes.

Assurance should combine pre-deployment review, runtime enforcement, and post-deployment evidence. Pre-deployment review checks server code, tool schemas, prompts, and policies. Runtime enforcement applies the policy gateway, context firewall, sandbox, and monitor. Post-deployment evidence uses audit logs, evaluation reports, incidents, and user feedback. No single phase is enough. A well-reviewed server can be compromised later; a strong policy can have an unanticipated exception; an audit log can reveal a failure that no static analysis predicted. AgentMesh-MCP is therefore an assurance loop rather than a one-time certification.

Finally, AgentMesh-MCP should be understood as compatible with multiple organizational maturity levels. A small team may implement a lightweight version using an allowlist, simple risk rules, file-based logs, and manual review. A regulated enterprise

may implement signed registries, formal policy engines, hardware-backed secrets, tamper-evident ledgers, automated red-team suites, and compliance dashboards. The architectural pattern remains the same: reason with the LLM, govern with deterministic controls, execute through MCP servers, observe continuously, and improve policy based on evidence. This is the path from experimental agents to dependable agentic infrastructure.

C. Assurance Cases, Compliance Mapping, and Organizational Adoption

An assurance case for AgentMesh-MCP should state a clear claim, identify evidence, and show why the evidence supports safe deployment. A minimal claim might be: this agent can execute read-only research tasks over approved MCP servers without leaking confidential data or invoking mutation tools. Evidence would include registry entries proving the servers are read-only, policy tests showing mutation calls are denied, context-firewall tests showing secret redaction, audit samples showing complete traces, and red-team tests showing resistance to indirect prompt injection. For a higher-risk claim, such as allowing an agent to modify staging infrastructure, the evidence must be stronger: rollback tests, dry-run comparisons, change-approval records, sandbox logs, and incident-response rehearsals. This assurance-case approach is useful because it prevents vague safety claims and forces the organization to connect architecture to measurable evidence.

Compliance mapping should not be treated as a paperwork exercise. NIST AI RMF functions such as govern, map, measure, and manage correspond naturally to AgentMesh-MCP components. The governance dashboard and policy registry support the govern function. Server inventory, tool classification, and data-flow analysis support the map function. Evaluation metrics, red-team testing, and audit completeness support the measure function. Risk-based approvals, rollback, incident response, and policy updates support the manage function [32]. ISO/IEC 42001 similarly requires an organizational management system for AI; AgentMesh-MCP provides runtime evidence that such a management system is not merely documented but operationalized [33].

A compliance-oriented deployment should maintain several artifacts. The first is an agent system card describing the agent's purpose, autonomy level, data sources, MCP servers, tool categories, model versions, human oversight points, and known limitations. The second is a tool risk register describing each MCP tool's side effects, data classification, owner, trust tier, and approval requirements. The third is a policy test suite that demonstrates allow, deny, approval, and sandbox outcomes. The fourth is an incident log and lessons-learned register. The fifth is an evaluation report that includes benign task performance and adversarial robustness. These artifacts allow internal reviewers, auditors, and regulators to inspect the system without relying on informal explanations from developers.

Organizational adoption should be incremental because agentic AI changes work practices. Users must understand that an agent is not a magic worker but a delegated system with bounded authority. Tool owners must learn to write precise schemas and side-effect descriptions. Security teams must learn to red-team prompts, memory, and tool metadata rather than only networks and code. Compliance teams must learn to read audit traces that include model outputs and tool calls. Product teams must design approval interfaces that are concise enough for real use.

AgentMesh-MCP is therefore partly a technical architecture and partly an organizational learning process.

A common adoption failure is treating the agent team as separate from platform engineering. In practice, secure agents require identity systems, secrets management, logging pipelines, deployment controls, policy engines, monitoring, and incident response. These are platform responsibilities. If agent developers build isolated prototypes with embedded credentials and local logs, the organization accumulates hidden risk. AgentMesh-MCP encourages a platform model: MCP server onboarding, policy enforcement, approval UI, and audit storage are shared services. Individual teams can build specialized agents on top of these services without rebuilding safety from scratch.

Another adoption failure is underestimating domain expertise. A legal-document agent, a cloud-operations agent, and a healthcare administrative agent may all use the same protocol layer, but their policies must be written with domain experts. The system must know which legal clauses are privileged, which infrastructure operations are production-impacting, and which healthcare fields are regulated. Purely generic policy cannot encode these distinctions. AgentMesh-MCP therefore separates the generic enforcement machinery from domain-specific policy packs. This allows reuse of infrastructure while preserving local control over risk definitions.

Economic evaluation should include avoided harm, not only automation savings. A direct tool-calling agent may appear cheaper because it requires less infrastructure and fewer approvals. However, a single data leak, cloud outage, compliance violation, or repository compromise can outweigh the cost of governance controls. The correct comparison is expected value under risk: productivity gains minus infrastructure cost, human review cost, incident probability, and incident severity. AgentMesh-MCP is justified when it preserves enough useful automation while reducing the probability or impact of high-severity failures. This framing helps decision-makers avoid both reckless adoption and excessive prohibition.

The framework also supports research transparency. Academic papers on agentic systems often report task success but omit operational details such as tool permissions, prompt versions, audit traces, and failure recovery. AgentMesh-MCP suggests a reporting checklist: describe the MCP servers used, tool side effects, authorization scope, human approval rules, context-filtering method, memory policy, attack corpus, and audit evidence. Such reporting would make experiments more reproducible and more relevant to real deployment. It would also help distinguish a model capability result from a systems engineering result.

Finally, AgentMesh-MCP can serve as a bridge between innovation and restraint. Organizations do not need to choose between banning agents and granting them broad autonomy. They can deploy agents in narrow domains, measure behavior, harden tools, expand scope gradually, and retain human authority over high-impact actions. This staged posture is especially important because model capabilities, MCP ecosystems, and regulations are all changing. A governed architecture allows organizations to adapt without redesigning from first principles each time a model, tool, or policy landscape changes.

D. Human Factors and Socio-Technical Risk

Human factors are central because AgentMesh-MCP deliberately includes people in approval, oversight, exception handling, and incident response. A poorly designed human approval step can create the appearance of governance without real control. If the approval screen shows a large JSON payload with no explanation, many users will approve by habit. If the screen hides the proposed side effect, users cannot evaluate risk. If approvals are too frequent, they become background noise. Therefore, the human approval gateway should be treated as a safety-critical interface. It should summarize intent, display the exact tool and arguments, state the risk rationale, highlight irreversible effects, show diffs for proposed changes, and provide clear approve, deny, edit, and request-information options.

Overtrust is a distinct failure mode. Users may assume that an agent is safe because it is fluent, because it cites sources, because it is embedded in an enterprise product, or because it has previously completed routine tasks. Overtrust can cause users to approve actions without reading them, accept generated explanations without verification, or delegate tasks that exceed the agent's validated scope. AgentMesh-MCP counters overtrust by making uncertainty and residual risk visible. The system should report when a plan depends on unverified retrieved content, when a tool result conflicts with memory, when a server has a low trust tier, or when the risk model is uncertain. Transparency should be operational, not merely rhetorical.

Undertrust can also reduce safety and usefulness. If users perceive the system as opaque or arbitrary, they may bypass it, create shadow tools, or pressure administrators to loosen policies. A governance framework must therefore explain denials and approvals in language that users can understand. When a tool call is blocked, the system should state whether the reason is missing permission, excessive data exposure, untrusted server status, high irreversibility, or a policy conflict. Good explanations turn governance into feedback. Users learn how to formulate safer goals, and tool owners learn how to improve schemas and permissions.

Organizational incentives matter. A team rewarded only for automation speed may disable approvals or broaden credentials. A team rewarded only for avoiding incidents may block useful autonomy. A mature governance program balances productivity and safety through measurable objectives: safe task completion, reduced manual toil, low violation rate, acceptable latency, audit completeness, and user satisfaction. AgentMesh-MCP provides the data needed for this balance, but leadership must decide how to use it. Metrics should be reviewed by cross-functional groups rather than by the agent development team alone.

Training and documentation are part of the control system. Users need to know which actions the agent is allowed to perform, which require approval, how to interpret approval cards, how to report suspicious behavior, and how to recover from mistakes. Developers need guidance on writing tool schemas, defining side effects, avoiding prompt-injection-prone descriptions, and testing policies. Security reviewers need playbooks for red-team scenarios. Without this organizational layer, even a technically sound architecture may fail in practice. AgentMesh-MCP therefore treats secure agentic AI as a socio-technical deployment whose success depends on people, process, and technology together.

A final socio-technical issue is accountability across delegation chains. In many deployments, a user gives a goal to an agent, the agent delegates a subtask to another agent, that agent calls an MCP server, and the server invokes a downstream API owned by a different team. If an incident occurs, it must be possible to identify which human authorized the goal, which agent proposed the action, which policy allowed it, which server executed it, and which system state changed. This is why AgentMesh-MCP models accountability as a chain of evidence rather than a single log entry. Delegation should preserve identity, purpose, scope, and provenance at each hop. This does not remove human responsibility, but it makes responsibility traceable enough for review, remediation, and institutional learning.

For this reason, the framework recommends periodic governance reviews that sample completed tasks and reconstruct their evidence chains end to end. These reviews should not be limited to failures. Successful tasks reveal whether the system is becoming dependent on overly broad permissions, fragile prompts, unreviewed memory, or informal human practices. Preventive review is cheaper than incident recovery and helps keep autonomy within the bounds originally justified by evidence.

This review discipline also creates a training corpus for future policy refinement and safer agent evaluation.

XIV. LIMITATIONS

AgentMesh-MCP does not eliminate hallucination. A model may still misunderstand a goal, infer an incorrect plan, or misinterpret a tool result. The architecture reduces the chance that hallucination becomes harmful external action, but it cannot guarantee correct reasoning. Stronger model calibration, formal task specifications, and tool result verification remain necessary research directions.

Long-term planning remains weak in current LLM agents. Agents can lose track of goals, repeat actions, or optimize local steps at the expense of global objectives. Graph-based planning and memory help, but they introduce complexity and new attack surfaces. AgentMesh-MCP constrains execution but does not solve the full problem of reliable long-horizon autonomy. This is why rollback, monitoring, and human control remain central.

Benchmark limitations are substantial. Existing agent benchmarks often fail to represent enterprise permissions, private data, adversarial documents, human approval, audit requirements, and rollback. A system can perform well on public web tasks yet fail in regulated workflows. The evaluation methodology proposed in this paper is a blueprint, not a completed benchmark. Creating standardized MCP security benchmarks is a major future task.

Unsafe tool use can still occur if policies are misconfigured. A policy gateway is only as good as its rules, data classifications, identity bindings, and enforcement points. Overly permissive tools, broad tokens, weak schemas, or incomplete logs can undermine the architecture. The framework requires disciplined operations: server registry maintenance, vulnerability tracking, policy testing, and incident drills.

Verifying agent behavior is difficult because the system is partly nondeterministic, context-sensitive, and dependent on external tools. Formal verification may be possible for constrained policies and tool schemas, but not for every natural-language reasoning step. AgentMesh-MCP therefore uses a hybrid strategy: deterministic enforcement around actions, empirical

red-team testing around behavior, and audit trails around accountability.

Governance complexity can slow adoption. Organizations may be tempted to deploy direct tool-calling agents because they are easier to build. AgentMesh-MCP requires additional infrastructure: registry, policy gateway, context firewall, audit ledger, approval UI, evaluation suite, and rollback mechanisms. The practical response is phased adoption. Start with read-only tools, limited domains, strong logging, and manual approvals; then increase autonomy only after measurement supports it.

Regulatory uncertainty remains. AI governance standards and laws are evolving, and their application to agentic systems is not fully settled. Human overtrust is another limitation. Users may assume that an agent with a formal architecture is safe in all circumstances. The correct posture is controlled trust: the system should communicate uncertainty, residual risk, and approval rationale rather than presenting autonomy as infallibility.

XV. FUTURE RESEARCH AGENDA

The first research priority is standardized MCP benchmarks. Such benchmarks should include tool discovery, tool invocation, malicious server metadata, prompt injection, memory poisoning, data leakage, authorization failures, rollback, and audit evaluation. They should represent realistic enterprise workflows rather than only toy tasks. A benchmark should score not only final success but also whether the agent completed the task through permitted actions.

The second priority is secure MCP server certification. A certification scheme could define server identity, schema quality, permission model, logging support, sandbox behavior, vulnerability disclosure, and provenance. Public registries would benefit from trust tiers that are machine-readable by agent hosts. Certification should not imply that a server is universally safe; it should describe verified properties and operational requirements.

The third priority is formal verification of agent actions. While full verification of LLM reasoning is unrealistic today, policy-gated tool invocation can be formalized. Researchers can verify that certain classes of tool call are impossible under given policies, that data-flow constraints are preserved, or that rollback plans exist for specified actions. Typed tool schemas, information-flow labels, and temporal logic policies are promising directions.

The fourth priority is memory safety. Future systems need principled memory provenance, retention, deletion, quarantine, and trust scoring. Memory should not be a monolithic vector store. It should be a governed subsystem with source labels, confidence, sensitivity, temporal validity, and review state. Agents should be able to explain why a memory item influenced a plan.

The fifth priority is multi-agent trust protocols. Multi-agent systems need agent identity, delegated authority, message signatures, belief provenance, and mechanisms for resolving conflicting claims. A specialist agent should not automatically inherit the authority of the orchestrator. Cross-agent communication should be treated as an internal supply chain of information and proposals.

The sixth priority is observability standards for agents. Logs should capture reasoning summaries, tool actions, policy decisions, and outcomes in interoperable formats. The community needs schemas for agent traces, tool provenance,

approval events, and incident records. Without observability standards, comparing agentic systems and auditing incidents will remain ad hoc.

The seventh priority is agent incident response. Organizations need playbooks for compromised agents, malicious MCP servers, leaked credentials, poisoned memory, unsafe tool executions, and audit-log gaps. Incident response should include token revocation, server quarantine, memory rollback, user notification, compliance reporting, and post-incident policy updates.

The final priority is safer autonomous tool use. The long-term goal is not to prevent all autonomy but to make autonomy conditional on demonstrated reliability, bounded authority, and recoverable action. Research should explore adaptive autonomy levels, calibrated risk scoring, human-agent shared control, and governance regulations that distinguish low-risk assistance from high-impact autonomous execution.

XVI. CONCLUSION

Agentic AI systems will shape software engineering, cloud operations, enterprise automation, research, education, cybersecurity, legal work, healthcare administration, and many other domains. Their value comes from the ability to connect reasoning with action. Their risk comes from the same source. LLM agents that can use MCP servers are more interoperable and more capable, but they also create new failure modes involving prompt injection, tool poisoning, malicious servers, credential misuse, data leakage, excessive agency, memory poisoning, and multi-agent compromise.

This paper proposed AgentMesh-MCP, a secure and governed framework for LLM agents and Model Context Protocol servers. The framework separates reasoning from execution, routes MCP calls through a policy and authorization gateway, filters context, scores risk, requires human approval for high-risk actions, monitors tool execution, records immutable audit events, supports rollback and recovery, and provides governance dashboards. It also formalized the system as a constrained transition process and supplied algorithms, evaluation metrics, experimental design, findings, use cases, limitations, and future research directions.

The main conclusion is that production-grade agentic AI should be designed as governed distributed infrastructure, not as a more elaborate chatbot. MCP can become a powerful interoperability layer for AI systems, but secure deployment requires policy, identity, observability, and human control around the protocol. AgentMesh-MCP offers one path toward that goal. Its broader significance is that it treats autonomy as an engineering variable to be measured, constrained, and governed. If implemented carefully, secure MCP-mediated agents can transform digital work while preserving accountability and institutional control.

REFERENCES

- [1] Anthropic, "Introducing the Model Context Protocol," Nov. 2024. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>
- [2] Model Context Protocol, "What is the Model Context Protocol (MCP)?" 2026. [Online]. Available: <https://modelcontextprotocol.io/docs/getting-started/intro>
- [3] Model Context Protocol, "Architecture overview," 2026. [Online]. Available: <https://modelcontextprotocol.io/docs/learn/architecture>

- [4] JSON-RPC Working Group, "JSON-RPC 2.0 Specification," 2013. [Online]. Available: <https://www.jsonrpc.org/specification>
- [5] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," arXiv:2210.03629, 2022.
- [6] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," arXiv:2302.04761, 2023.
- [7] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," arXiv:2303.11366, 2023.
- [8] G. Wang et al., "Voyager: An open-ended embodied agent with large language models," arXiv:2305.16291, 2023.
- [9] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework," arXiv:2308.08155, 2023.
- [10] J. S. Park et al., "Generative agents: Interactive simulacra of human behavior," in Proc. UIST, 2023.
- [11] R. Nakano et al., "WebGPT: Browser-assisted question-answering with human feedback," arXiv:2112.09332, 2021.
- [12] Y. Qin et al., "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," arXiv:2307.16789, 2023.
- [13] X. Liu et al., "AgentBench: Evaluating LLMs as agents," arXiv:2308.03688, 2023.
- [14] Z. Ma et al., "AgentBoard: An analytical evaluation board of multi-turn LLM agents," arXiv:2401.13178, 2024.
- [15] S. Zhou et al., "WebArena: A realistic web environment for building autonomous agents," arXiv:2307.13854, 2023.
- [16] S. Yao et al., "WebShop: Towards scalable real-world web interaction with grounded language agents," in Proc. NeurIPS, 2022.
- [17] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.
- [18] E. Karpas et al., "MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning," arXiv:2205.00445, 2022.
- [19] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in Proc. NeurIPS, 2022.
- [20] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," in Proc. ICLR, 2023.
- [21] T. Kojima et al., "Large language models are zero-shot reasoners," in Proc. NeurIPS, 2022.
- [22] T. B. Brown et al., "Language models are few-shot learners," in Proc. NeurIPS, 2020.
- [23] R. Bommasani et al., "On the opportunities and risks of foundation models," arXiv:2108.07258, 2021.
- [24] OpenAI, "Function calling and tool calling developer documentation," 2026. [Online]. Available: <https://platform.openai.com/docs/guides/function-calling>
- [25] LangChain, "LangChain documentation: Agents and tools," 2026. [Online]. Available: <https://docs.langchain.com/>
- [26] LangChain, "LangGraph overview," 2026. [Online]. Available: <https://docs.langchain.com/langgraph>
- [27] Microsoft, "Semantic Kernel documentation," 2026. [Online]. Available: <https://learn.microsoft.com/semantic-kernel/>
- [28] OWASP Foundation, "OWASP Top 10 for LLM Applications / GenAI Security Project," 2025. [Online]. Available: <https://genai.owasp.org/>
- [29] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. ACM AISec, 2023.
- [30] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," arXiv:2211.09527, 2022.
- [31] Y. Liu et al., "Prompt injection attacks and defenses in LLM-integrated applications," arXiv:2310.12815, 2023.
- [32] E. Tabassi, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, 2023.
- [33] ISO/IEC, "ISO/IEC 42001:2023 Information technology - Artificial intelligence - Management system," 2023.
- [34] ISO/IEC, "ISO/IEC 23894:2023 Information technology - Artificial intelligence - Guidance on risk management," 2023.
- [35] European Union, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," Official Journal of the European Union, 2024.
- [36] D. Hardt, "The OAuth 2.0 Authorization Framework," RFC 6749, IETF, 2012.
- [37] OpenID Foundation, "OpenID Connect Core 1.0 incorporating errata set 2," 2023.
- [38] NIST, "Security and Privacy Controls for Information Systems and Organizations," NIST SP 800-53 Rev. 5, 2020.
- [39] S. Rose et al., "Zero Trust Architecture," NIST SP 800-207, 2020.
- [40] MITRE, "Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS)," 2026. [Online]. Available: <https://atlas.mitre.org/>
- [41] NIST, "Secure Software Development Framework (SSDF) Version 1.1," NIST SP 800-218, 2022.
- [42] OpenSSF, "Supply-chain Levels for Software Artifacts (SLSA) specification," 2026. [Online]. Available: <https://slsa.dev/>
- [43] L. Weidinger et al., "Ethical and social risks of harm from language models," arXiv:2112.04359, 2021.
- [44] D. Hendrycks et al., "Unsolved problems in ML safety," arXiv:2109.13916, 2021.
- [45] P. Manakul, A. Liusie, and M. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in Proc. EMNLP, 2023.
- [46] A. Madaan et al., "Self-Refine: Iterative refinement with self-feedback," in Proc. NeurIPS, 2023.
- [47] OpenTelemetry, "OpenTelemetry documentation," 2026. [Online]. Available: <https://opentelemetry.io/docs/>
- [48] W3C, "PROV-DM: The PROV Data Model," W3C Recommendation, 2013.
- [49] A. Narayanan and S. Kapoor, "AI Snake Oil: What Artificial Intelligence Can Do, What It Cannot, and How to Tell the Difference," Princeton University Press, 2024.
- [50] CISA, NSA, FBI, and international partners, "Guidelines for Secure AI System Development," 2023.